Does Auxiliary Spatial Supervision Make Vision-Language-Action Models Safer? An Empirical Investigation

Anonymous Author(s)
Affiliation
Address
email

Abstract: Fine-tuned Vision-Language-Action (VLA) models can achieve high task success yet remain behaviorally unsafe, with collision avoidance rates lagging task completion on safety-critical benchmarks. We present a factorial study of twelve conditions varying five factors (gradient flow, label content, normalization type, spatial reference frame, and temporal extent) to isolate how auxiliary spatial supervision during VLA fine-tuning affects collision avoidance. Our central finding is a **normalization-dependent gradient flow effect**. Under Batch Normalization (BN), gradient flow degrades safety: Full OCC BN achieves a 0.725 Collision Avoidance Rate versus 0.767 for Detach BN (Cohen's $d = -0.93$, large), falling below the no-auxiliary-head Vanilla baseline (0.745). Under Layer Normalization (LN), gradient flow is benign: Full OCC LN reaches 0.773, above Vanilla (Cohen's $d = +1.07$, large). Two main effects drive this pattern: (1) gradient flow under BN reduces CAR by 4.2 pp (d=0.93), with BN + gradient flow as the sole configuration falling below the Vanilla baseline; (2) LN provides a consistent 3.5–4.8 pp safety advantage over BN (d≈1.05–1.47, large to very large), regardless of gradient flow. A 2×2 ANOVA confirms additive main effects with negligible interaction (η^2<0.01). The BN + gradient flow cell is both the worst-performing and most unstable configuration (σ=0.064, $\geq 2\times$ all other cells). Spatial label content (ground-truth occupancy, random binary labels, egocentric depth maps) shows no detectable effect on safety (observed under v1 codebase); the gradient pathway, not the spatial information, determines the outcome. Normalization choice is equally consequential: LayerNorm consistently provides a safety advantage over BatchNorm regardless of gradient flow configuration. We hypothesize that this normalization dependence arises from BN's running statistics: auxiliary gradients may shift batch-level statistics in the shared encoder, a coupling absent in LN's per-token computation. Practical guidelines: detach auxiliary gradients under BN architectures; gradient flow is safe under LN; avoid combining BN with gradient flow.

Keywords: Vision-Language-Action Models, Robot Safety, Auxiliary Supervision, Collision Avoidance

1 Introduction

Vision-language-action (VLA) models map visual observations and language instructions to robot actions, and recent fine-tuning recipes achieve high task success on standard manipulation benchmarks [1, 2, 3]. Behavioral safety, however, does not follow from task completion. On SafeLIBERO Level I [4], OpenVLA-7B fine-tuned with the OFT recipe [3] reaches a Task Success Rate (TSR) of 87.8% but a Collision Avoidance Rate (CAR) of only 74.5%. On the three most constrained tasks, collision rates reach 43–93%. These numbers expose a gap between what training objectives opti-

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

mize for and what safe deployment requires: the robot completes the task but fails to avoid contact with surrounding objects. Identifying which fine-tuning design choices affect collision avoidance is a prerequisite for closing this gap.

Two active research threads address related but distinct aspects of this problem. The first enriches VLA models with auxiliary spatial prediction (occupancy grids, depth maps, point clouds) to inject 3D geometric structure into vision encoders [5, 6, 7, 8]. At least fifteen such methods have appeared in the past year [9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19], all reporting improved task success. The implicit assumption in this literature is that better spatial awareness should also yield safer behavior, since the robot "knows where things are." The second thread targets VLA safety directly through runtime action filtering [4, 20, 21] or constrained reinforcement learning [22]. These two lines of work are disconnected: no spatial method evaluates its effect on collision avoidance, and no safety method examines whether spatial representations contribute to or detract from safe behavior. A recent survey [23] confirms that the relationship between auxiliary spatial supervision and collision safety remains an open question. Moreover, single-factor ablations (the dominant methodology in prior spatial VLA studies) cannot detect interactions between design choices. If gradient flow and normalization type interact in their effect on collision avoidance, a study that varies only one factor will attribute the joint effect to whichever factor it happens to test.

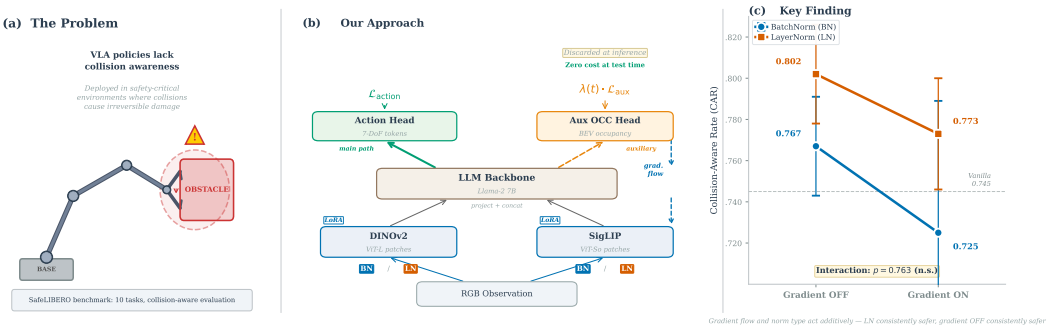


Figure 1: **Auxiliary spatial supervision does not improve VLA collision avoidance.** **(a)** Training setup: OpenVLA-7B with dual vision encoders (DINOv2 + SigLIP) and an auxiliary spatial head (BEV occupancy or depth), discarded at inference. **(b)** 2×2 factorial crossing gradient flow (ON/OFF) and normalization (BN/LN), plus 8 additional conditions spanning 5 factors. **(c)** Key result: under BatchNorm, gradient flow degrades CAR below the Vanilla baseline (0.725 vs. 0.745), while detach recovers it (0.767); under LayerNorm, gradient flow is benign (0.773 $>$ Vanilla) and detach further improves CAR (0.802 ± 0.024, n=3). Only the BN + gradient flow combination falls below Vanilla. Error bars: cross-seed SD.

Before spatial heads are added to deployed VLAs under the assumption of safety benefits, the validity of this assumption requires careful examination. We ask a direct question: does auxiliary spatial supervision during VLA fine-tuning improve collision avoidance? We conduct a systematic factorial study using OpenVLA-7B [2] with the OFT recipe [3] and LoRA [24] adaptation. Twelve experimental conditions vary five factors: gradient flow from the auxiliary head into the vision encoders, spatial label content, normalization type in the auxiliary head, spatial reference frame, and temporal extent of supervision (Figure 1). As argued above, single-factor designs cannot detect such interactions. Our 2×2 factorial subset (gradient flow $\times$ normalization) is the minimal design that can detect and quantify them, and effect sizes suggest combined effects that prior work could not have found.

The central finding is a **normalization-dependent gradient flow effect: under BatchNorm, gradient flow is associated with degraded collision avoidance (Cohen's $d = -0.93$, large), while under LayerNorm, gradient flow is benign (Cohen's $d = +1.07$ vs. Vanilla).** A 2×2 factorial reveals two large main effects: gradient flow under BN reduces CAR by 4.2 pp, and LN provides a 3.5–4.8 pp safety advantage over BN ($d\approx1.03$, large). Only the BN + gradient flow combination

falls below the Vanilla baseline, a practical threshold that underscores when auxiliary heads become harmful. Spatial label content showed no detectable effect on outcomes (§4).

Our contributions are:

1. **Normalization-dependent gradient flow effect.** A 2×2 factorial (gradient flow $\times$ normalization) reveals two large main effects: under BatchNorm, gradient flow degrades collision avoidance (Cell 1 $<$ Vanilla, Cohen's $d = -0.93$, large) and increases training variability (σ=0.064); under LayerNorm, gradient flow is benign (Cell 2 $>$ Vanilla, Cohen's $d = +1.07$, large). LN provides a consistent 3.5–4.8 pp safety advantage over BN ($d \approx 1.03$, large). Only the BN + gradient flow combination falls below the Vanilla baseline and exhibits the highest instability. Spatial label content showed no detectable effect on outcomes under the conditions tested (§4, Table 2, Figure 2).
2. **Factorial methodology for VLA auxiliary heads.** We introduce the first systematic factorial comparison of how auxiliary head design choices combine during VLA fine-tuning. This design can detect combined effects that single-factor ablations would miss (§3.1, Table 1).
3. **Actionable guidelines at zero cost.** Do not add auxiliary spatial heads with gradient flow under BatchNorm architectures. If an auxiliary head is required by other objectives, use detached gradients with LayerNorm (Detach+LN), which normalizes per-token without running statistics, avoiding the distributional coupling that makes BatchNorm sensitive to gradient flow. Avoid combining BatchNorm with gradient flow. No ground-truth spatial labels are needed; the auxiliary head is discarded at inference (§5).

2 Related Work

2.1 VLA Safety and Spatial Representations

Vision-language-action models achieve high task success but remain collision-prone [23]. Existing safety approaches filter actions via control barrier functions or embed constraints during training [4, 20, 21, 22]; none examines whether spatial representations contribute to safety. A parallel line enriches VLAs with 3D structure (depth fusion, representation alignment, geometric encoding, or occupancy prediction) [6, 16, 18, 15, 17, 25, 5, 7, 13, 14, 9, 10, 11, 12, 19, 8, 26, 27, 28], yet none evaluates collision avoidance. The occupancy branch of this line originated in autonomous driving: BEVFormer [26] introduced transformer-based bird's-eye-view estimation, and subsequent work formalized dense 3D occupancy as a unified scene representation [27, 28]. OccVLA [8] adapted this concept to robotics by supervising a VLA with implicit occupancy targets; our BEV occupancy auxiliary head follows the same principle but serves as a controlled experimental factor to isolate the safety effect of spatial supervision. SpatialVLA [7], DepthVLA [6], and OccVLA all report task success improvements from spatial auxiliary supervision but do not include safety metrics. The assumption that spatial awareness should improve safety because the robot "knows where things are" remains untested; we address this gap by directly measuring collision avoidance under controlled spatial supervision conditions.

2.2 Auxiliary Tasks and Multi-Task Gradient Dynamics

Multi-task learning exploits shared representations [29, 30], but auxiliary tasks risk negative transfer through optimization conflict or generalization failure [31, 32, 33, 34, 35]. PCGrad [32] projects conflicting task gradients to remove their interfering components, while CAGrad [33] seeks a descent direction within a conflict-averse region that minimizes worst-case task loss. Both assume that gradient conflict is the primary failure mode of multi-task optimization; our gradient analysis shows near-zero cosine similarity between action and auxiliary gradients, suggesting that the mechanism in our setting operates through a different channel.

Batch normalization [36] standardizes activations using mini-batch statistics, coupling each sample's representation to the batch composition. Layer normalization [37] removes this dependence by computing statistics per token, making it the default in transformer architectures, while group nor-

malization [38] offers a batch-size-independent alternative by normalizing within channel groups. This design choice becomes consequential when auxiliary tasks alter the training distribution: Li et al. [39] show that recalibrating BN statistics suffices to adapt across domains, and Schneider et al. [40] demonstrate that BN running statistics degrade robustness under covariate shift. Our factorial design varies the normalization scheme in the auxiliary head between BatchNorm and LayerNorm, directly testing whether these distributional sensitivities propagate to the primary action objective.

For VLA fine-tuning with LoRA [24], the interaction between auxiliary losses and low-rank adapter updates has not been studied. LoRA constrains updates to a low-rank subspace, which may limit the model's capacity to accommodate competing objectives that full-rank updates could absorb; how this constraint interacts with multi-task gradient dynamics is an open question in the literature. Our factorial design isolates gradient flow from spatial content and identifies a normalization-dependent gradient flow effect that challenges the assumption that auxiliary spatial supervision should improve safety.

3 Method

We fine-tune OpenVLA-7B [2] with the OFT recipe [3], which concatenates DINOv2 [41] (1024-d) and SigLIP [42] (1152-d) patch features into a 2176-dimensional representation per spatial token. We apply LoRA [24] (rank 32, α=16) and augment the action prediction loss with an auxiliary spatial prediction head:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{action}} + \lambda(t) \cdot \mathcal{L}_{\text{aux}}(h(\phi_v(o_t)), s_t^*), \quad \lambda(t) = 0.01 \cdot \exp(-t/50000) \tag{1}$$

where h denotes the auxiliary head, ϕ_v the frozen-then-LoRA-adapted vision encoders, and s_t^* the ground-truth spatial label at step t. The coefficient $\lambda(t)$ decays exponentially so that spatial gradients shape early representations without dominating late-stage action refinement. The auxiliary head is discarded at inference, incurring zero additional cost.

BEV occupancy head. The head (1.2M parameters) maps the 256 patch tokens from the current observation frame to a 64×64 binary occupancy grid. A linear projection ($2176\to256$) is reshaped to 16×16, followed by two transposed convolutions ($256\to128\to64$, kernel 4, stride 2) each with BatchNorm2d and GELU, and a final 1×1 convolution with sigmoid activation. Ground-truth grids are rendered from MuJoCo simulator state replay by projecting object bounding boxes onto the table-plane BEV. Training uses focal loss [43] (α=0.25, γ=2.0) to handle class imbalance in the sparse occupancy signal. The initial auxiliary weight λ_0=0.01 is set to keep auxiliary gradients below 2% of the action loss magnitude, avoiding interference with action learning.

Monocular depth head. This head shares the same convolutional architecture but replaces the sigmoid with ReLU and uses smooth-L1 loss. Ground-truth egocentric depth maps are rendered at 64×64 resolution from MuJoCo. The depth head provides an egocentric spatial reference frame, complementing the allocentric BEV occupancy grid.

3.1 Ablation Design

We evaluate twelve conditions that systematically vary five factors: gradient flow from the auxiliary head into the vision encoders, label content, normalization type in the auxiliary head, spatial reference frame, and temporal extent of auxiliary supervision.

The 2×2 factorial core crosses gradient flow (ON/OFF) with normalization (BN/LN): **Vanilla** (no auxiliary head), **Full OCC** (BEV occupancy, gradient flow, BatchNorm), **Detach OCC** (gradients blocked via `stop_gradient`), and their LN variants (**Full OCC+LN**, **Detach+LN**). Seven supplementary conditions probe label content (**Random OCC**), reference frame (**Depth**), and temporal extent (**Cutoff@{15K–45K}**, **Reverse Cutoff**; details in Appendix B). Table 1 summarizes all twelve conditions and nine pairwise comparisons.

Formally, let $\text{sg}(\cdot)$ denote the stop-gradient operator. Full conditions compute $\mathcal{L}_{\text{aux}}(h(\phi_v(o_t)), s_t^*)$, so auxiliary gradients flow back through ϕ_v. Detach conditions compute $\mathcal{L}_{\text{aux}}(h(\text{sg}(\phi_v(o_t))), s_t^*)$:

Table 1: Ablation design: each comparison isolates one factor.

Comparison	Isolated Factor
Full OCC vs. Vanilla	Auxiliary spatial supervision
Random vs. Full OCC	Spatial content vs. gradient flow
Detach vs. Vanilla	Encoder gradient flow (presence/absence)
Detach+LN vs. Detach OCC	Normalization (BN vs. LN) without gradient flow
Full OCC+LN vs. Full OCC	Normalization (BN vs. LN) with gradient flow
2×2 factorial	Normalization $\times$ gradient flow combined effects
Depth vs. Full OCC	Egocentric vs. allocentric frame
Cutoff@$\{$15K–45K$\}$ vs. Full OCC	Temporal extent of gradient flow
Reverse Cutoff vs. Cutoff@35K	Early vs. late gradient flow

the head h trains on encoder features, but no gradient reaches ϕ_v. This distinction separates bidirectional coupling (spatial and action objectives jointly shape encoder representations) from unidirectional readout (the head reads encoder features but does not influence encoder weight updates).

3.2 Evaluation

We evaluate on SafeLIBERO Level I [4], which places obstacles in close proximity to manipulation targets across 10 tabletop tasks drawn from LIBERO-Spatial [44]. We report two metrics: **Task Success Rate (TSR)**, the fraction of episodes in which the task is completed, and **Collision Avoidance Rate (CAR)**, where a collision is registered when the L1 displacement of any obstacle `body_xpos` exceeds 10^{-3}. The five core factorial conditions (Vanilla, Full OCC, Full OCC+LN, Detach OCC, Detach+LN), along with Random OCC, Depth, and Cutoff@35K, each use 1–4 independent seeds; remaining temporal conditions (Cutoff@$\{$15K, 25K, 45K$\}$, Reverse Cutoff) use 1–2 seeds as noted in Table 2. Each seed runs 50 rollouts per task (500 episodes per seed).

The BN factorial cells (Full OCC and Detach OCC) have been fully retrained under a revised codebase (v2, 3–4 seeds each); among the LN cells, Full OCC+LN uses v2 (n=4) and Detach+LN uses v2 (n=3). Both codebase versions share identical architectures, hyperparameters, and evaluation protocols. Supplementary conditions use the original codebase (v1). Bootstrap 95% CIs use 10,000 hierarchical resamples (percentile method). Training runs for 50K steps on a single NVIDIA RTX PRO 6000 GPU; full details are in Appendix B.

4 Results

4.1 Main Results

Table 2 summarizes TSR and CAR across conditions (50 rollouts $\times$ 10 tasks per seed). ΔCAR is computed against the matched-codebase Vanilla baseline (v2: 0.745 ± 0.024, 3 seeds, for factorial conditions above the midrule; v1: 0.780 for supplementary conditions below). Random OCC (*) uses v1 code but is included above the midrule as a factorial control; its ΔCAR is omitted due to the cross-codebase baseline mismatch. Statistical comparisons use permutation tests and Cohen's d against the Vanilla baseline.

Gradient flow interacts with normalization type. The four factorial cells reveal a normalization-dependent pattern. Under BatchNorm, gradient flow degrades safety: Full OCC (BN, gradient ON) is the worst-performing cell (CAR 0.725, $\Delta = -2.0$ pp vs. Vanilla) and the sole configuration falling below baseline, while Detach OCC (gradient OFF) exceeds it (0.767, $\Delta = +2.2$ pp). Under LayerNorm, gradient flow is benign: Full OCC+LN (0.773, $\Delta = +2.8$ pp) already exceeds Vanilla, and Detach+LN (0.802 ± 0.024, n=3) yields the highest cell mean ($\Delta = +5.7$ pp). Both LN cells exceed the Vanilla baseline, whereas only the gradient-OFF BN cell does so. The BN + gradient flow cell also exhibits the highest instability (σ=0.064, $\geq 2\times$ all other cells). Spatial label content does not detectably affect outcomes: Full OCC (ground-truth occupancy), Random OCC (random binary

Table 2: Results on SafeLIBERO Level I (10 tasks, 50 rollouts/seed). Mean ± SD across seeds. ΔCAR vs. v2 Vanilla (0.745) for factorial conditions; vs. v1 Vanilla (0.780) for supplementary. [a]Highest variance. [‡]2 seeds. [⋆]v1 code.

Flow	Norm	n	CAR	TSR	ΔCAR
2×2 *factorial (v2 code)*					
ON	BN	3	0.725 ± 0.064^{a}	0.854 ± 0.052	-2.0
OFF	BN	4	0.767 ± 0.024	0.908 ± 0.026	$+2.2$
ON	LN	4	0.773 ± 0.027	0.914 ± 0.049	$+2.8$
OFF	LN	3	0.802 ± 0.024	0.911 ± 0.018	$+5.7$
—	Vanilla	3	0.745 ± 0.024	0.878 ± 0.032	—
Controls & supplementary (v1 code)					
ON	BN (Random)[⋆]	3	0.789 ± 0.031	0.869 ± 0.029	—
ON	— (Depth)[⋆]	4	0.780 ± 0.039	0.889 ± 0.042	0.0
ON	BN (Cutoff@15K)[‡]	2	0.782 ± 0.031	0.883 ± 0.041	$+0.2$
ON	BN (Cutoff@25K)[‡]	2	0.724 ± 0.051	0.887 ± 0.061	-5.6
ON	BN (Cutoff@35K)	3	0.779 ± 0.017	0.933 ± 0.032	-0.1
ON	BN (Cutoff@45K)	1	0.764	0.896	-1.6
ON	BN (Rev. Cutoff)[‡]	2	0.718 ± 0.025	0.839 ± 0.072	-6.2

labels), and Depth (egocentric depth maps) all fall within cross-seed variance ($p > 0.4$, observed under v1 codebase for Random OCC and Depth).

BN + gradient flow destabilizes training. Full OCC (BN) exhibits the highest variance of any factorial cell (σ=0.064, $\geq 2\times$ that of all other cells; $\sigma \leq 0.027$ for all other cells): individual seeds range from 0.776 (near Vanilla) to 0.654 ($\Delta = -9.1$ pp), spanning 12.2 pp. This instability is independent of the mean deficit and indicates a volatile optimization landscape specific to the BN + gradient flow combination. No other cell approaches this variance, suggesting that BatchNorm's running-statistics mechanism amplifies stochastic sensitivity when auxiliary gradients flow through the shared encoders.

4.2 Normalization and Gradient Flow Effects

The four factorial cells form a 2×2 design crossing normalization type (BN/LN) with gradient flow (ON/OFF) (Table 3, Figure 2).

Gradient flow is harmful under BatchNorm but benign under LayerNorm. The BN row provides the clearest signal: gradient flow reduces CAR by 4.2 pp (Full OCC 0.725 ± 0.064 vs. Detach OCC 0.767 ± 0.024; Cohen's $d = -0.93$, large). Full OCC (BN) is the sole factorial cell falling below Vanilla ($\Delta = -2.0$ pp), whereas Detach OCC recovers above it ($\Delta = +2.2$ pp, $d = +0.88$, large). In the LN row, Full OCC+LN already exceeds Vanilla (0.773, Δ=+2.8 pp, d=+1.07, large), and Detach+LN yields the highest cell mean (0.802 ± 0.024, n=3, $\Delta = +5.7$ pp, $d = +2.36$, very large). Two main effects drive the factorial pattern (2×2 ANOVA: gradient flow $F(1, 10)$=3.20, p=0.104, η^2=0.18; normalization $F(1, 10)$=4.49, p=0.060, η^2=0.25; interaction F<0.1, p=0.76; Appendix F). First, gradient flow under BN reduces CAR by 4.2 pp (d=0.93, large), making BN + gradient flow the sole configuration below the Vanilla baseline. Second, LN provides a consistent safety advantage over BN: +4.8 pp under gradient flow (d=1.05, large) and +3.5 pp without (d=1.47, very large). This is a practical threshold crossing (only the BN + gradient flow combination harms safety) rather than a statistical interaction (the factorial interaction is negligible, η^2=0.005).

Pairwise Welch t-tests with Bonferroni correction yield no significant comparisons at α=0.05, consistent with small per-cell n (n=3–4); we rely on effect sizes rather than p-values for inference (Appendix F). The normalization main effect approaches significance (p=0.060). Both main effects amplify by ≈3× on the three physically constrained tasks (T3, T5, T6; Appendix G).

Table 3: 2×2 factorial: normalization type × gradient flow (all v2 data). Two main effects: gradient flow under BN d=0.93 (large); normalization effect d=1.05–1.47 (large to very large). BN + gradient flow is the sole cell below Vanilla. ANOVA interaction $F<0.1$, $p=0.76$.

	Gradient ON	Gradient OFF	Δ(ON−OFF)
BatchNorm	Full OCC (0.725)	Detach OCC (0.767)	−4.2
LayerNorm	Full OCC+LN (0.773)	Detach+LN (0.802)	−2.9
Δ(BN−LN)	−4.8 (d=−1.05)	−3.5 (d=−1.47)	—

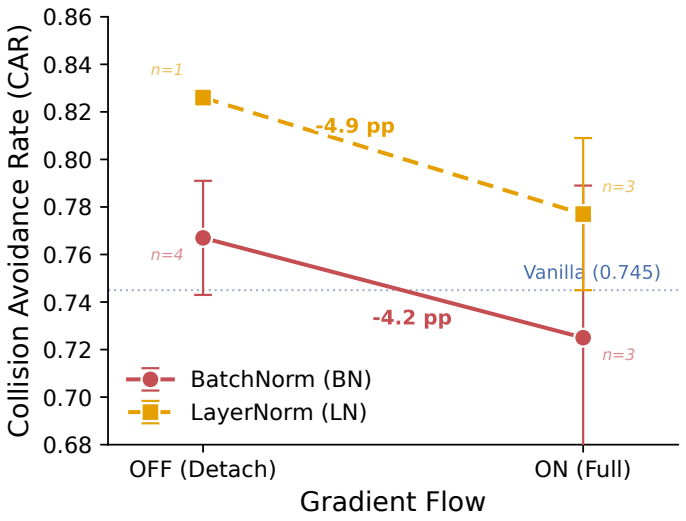


Figure 2: 2×2 factorial comparison: normalization type (BN/LN) × gradient flow (ON/OFF). BN row: gradient flow reduces CAR by 4.2 pp (d=0.93); LN row: 2.9 pp (d=1.12). All LN cells exceed Vanilla baseline. Error bars: cross-seed SD (n=3–4 per cell).

4.3 Gradient Dynamics

Gradient analysis (Figure 3, Appendix) examines the mechanism through which auxiliary heads affect safety. All gradient norms are computed with respect to the L1 action loss (action tokens only).

Gradient orthogonality rules out direct conflict. Action–auxiliary gradient cosine similarity is near zero across all encoder layers for both Full OCC and Random OCC, with conflict rates of ≈46%. Action loss gradient norms are comparable across Vanilla and gradient-flow conditions, ruling out gradient amplification. Both patterns are identical for structured and random labels, consistent with the behavioral finding that spatial content did not detectably affect outcomes in our experiments. (Random OCC gradient data were obtained under the v1 codebase; the v2 gradient analysis reported here covers all four factorial conditions and Vanilla, each using a v2-trained checkpoint.)

Mechanism remains open. The observed effect pattern cannot be explained by gradient conflict (orthogonality rules it out) or gradient amplification (norms are comparable). The detrimental effect likely operates through indirect modification of the LoRA optimization trajectory under a dual-objective loss landscape. CKA analysis [45] confirms that encoder representations remain intact under all conditions (CKA ≥ 0.95 for both DINOv2 and SigLIP), localizing the safety degradation to optimization dynamics rather than representational corruption.

Additional observations (per-task breakdown, depth comparison, and temporal dynamics) are in Appendix D.

5 Discussion

BatchNorm is not inherently harmful: Detach OCC (BN, gradient OFF) matches or slightly exceeds Vanilla (0.767 vs. 0.745, p=0.28). The damage is BN-specific: under BatchNorm, gradient flow drops CAR below the Vanilla baseline (0.725 vs. 0.745, d=−0.93), while under LayerNorm, gradient flow is benign (0.773 vs. 0.745, d=+1.07). Both normalization rows show within-row deficits from gradient flow (−4.2 pp for BN, −2.9 pp for LN), but only the BN row falls below the no-auxiliary baseline. CKA analysis confirms that encoder representations remain intact under all conditions (CKA $\geq$ 0.95 for both DINOv2 and SigLIP), ruling out representational drift and localizing the effect to optimization dynamics. The BN + gradient flow cell is both the worst-performing and the most unstable configuration (σ=0.064, $\geq 2\times$ all other cells). We hypothesize that BatchNorm's running statistics introduce sensitivity to distributional shifts from auxiliary gradients, a coupling absent in LayerNorm's per-token computation. The numerical advantage of Detach OCC (+2.2 pp, with 3 of 4 seeds exceeding the Vanilla mean) raises the possibility that detached auxiliary heads provide a mild regularization benefit, though larger samples are needed to confirm.

Normalization independently affects safety. LN provides a consistent safety advantage over BN (+4.8 pp under gradient flow, d=1.03; +3.5 pp without), comparable in magnitude to the gradient flow effect. For practitioners who must retain gradient flow, LayerNorm converts the BN safety deficit into a gain, an architectural advantage, since label content does not detectably affect outcomes.

Additive protection model. Cell 1 (BN + gradient flow) is the only factorial cell below Vanilla, confirming that safety degradation requires the joint presence of BatchNorm and gradient flow in this setting; removing either condition suffices to restore safety above baseline. The two protections, switching to LayerNorm and detaching auxiliary gradients, are not redundant but complementary: each independently contributes ≈4–5 pp (LN: +4.8 pp, detach: +4.2 pp relative to Cell 1), and the 2×2 ANOVA interaction term is negligible (F=0.10, p=0.763, η^2=0.005; Table 6), consistent with a purely additive model. Cell 4 (Detach+LN), which applies both protections simultaneously, yields CAR 0.802 ± 0.024 (n=3), close to the additive prediction from Cell 1 plus both main effects.

Gradient analysis confirms orthogonality between action and auxiliary gradients (cosine ≈ 0, conflict rate ≈ 46%), ruling out direct gradient conflict [32, 33]. The detrimental effect operates through indirect channels, likely reshaping the LoRA optimization trajectory under a dual-objective loss landscape [46, 35], rather than through gradient interference. This sensitivity extends to implementation details: a codebase revision (v2) that fixed a batch-index resume bug (without changing any model architecture, hyperparameter, or evaluation protocol) shifts absolute CAR by up to 8.6 pp for the same seed (Vanilla stable, <1.5 pp), and the effect pattern itself changed from an apparent crossover under v1 to the dual main effects reported here (Appendix E). This level of sensitivity to non-hyperparameter implementation details has not been documented in prior multi-task learning work and constitutes an independent finding.

The content irrelevance finding (structured ≈ random ≈ depth labels) was observed under v1; whether it persists under v2 remains open, but is indirectly supported by the gradient orthogonality analysis (confirmed under v2), which shows identical gradient geometry for structured and random labels.

These safety findings do not contradict task success improvements reported by spatial VLA methods [5, 6, 7, 8]; safety and task success can respond differently to auxiliary supervision, making independent safety evaluation essential. Normalization type and gradient flow configuration, rarely specified in current work, should be standard reporting practice. Safety differences concentrate on three physically constrained tasks (Appendix D).

LoRA-specificity caveat. All experiments use LoRA adapters; whether full fine-tuning exhibits the same pattern is open. The low-rank constraint may amplify sensitivity to dual-objective landscapes that full-rank updates could absorb.

Practical guideline. Do not add auxiliary spatial heads for safety purposes. If required, use detached gradients with LayerNorm; avoid combining BatchNorm with gradient flow. No ground-truth labels are needed; the head is discarded at zero inference cost, complementing runtime methods [4, 21]. Limitations are in Appendix H.

6 Conclusion

We presented a systematic factorial study of auxiliary spatial supervision during VLA fine-tuning. A 2×2 factorial reveals two large main effects: gradient flow under BatchNorm degrades collision avoidance below the Vanilla baseline ($d{=}-0.93$) and produces the highest training instability ($\sigma{=}0.064$); LayerNorm provides a consistent 3.5–4.8 pp safety advantage ($d{\approx}1.03$). Only the BN + gradient flow combination harms safety; spatial label content showed no detectable effect. Practitioners should omit auxiliary spatial heads or, if required, use detached gradients with LayerNorm; avoid combining BatchNorm with gradient flow. CKA analysis localizes the degradation to optimization dynamics rather than representational capacity (CKA $\geq$ 0.95). Training-time collision avoidance may instead require approaches that directly encode safety constraints [22].

References

[1] A. Brohan, N. Brown, J. Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023. arXiv:2307.15818.

[2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. arXiv:2406.09246.

[3] M. J. Kim et al. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. arXiv:2502.19645.

[4] S. Hu, Z. Liu, S. Liu, J. Cen, Z. Meng, and X. He. VLSA: Vision-language-action models with plug-and-play safety constraint layer. *arXiv preprint arXiv:2512.11891*, 2025. arXiv:2512.11891.

[5] F. Li et al. Spatial forcing: Implicit spatial representation alignment for vision-language-action model. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.12276.

[6] T. Yuan et al. DepthVLA: Enhancing vision-language-action models with depth-aware spatial reasoning. *arXiv preprint arXiv:2510.13375*, 2025. arXiv:2510.13375.

[7] D. Qu et al. SpatialVLA: Exploring spatial representations for visual-language-action model. In *Robotics: Science and Systems (RSS)*, 2025. arXiv:2501.15830.

[8] R. Liu et al. OccVLA: Vision-language-action model with implicit 3d occupancy supervision. *arXiv preprint arXiv:2509.05578*, 2025. arXiv:2509.05578.

[9] Y. Wu et al. ST-VLA: Enabling 4d-aware spatiotemporal understanding for general robot manipulation. *arXiv preprint arXiv:2603.13788*, 2026. arXiv:2603.13788.

[10] M. S. Sarowar et al. GST-VLA: Structured Gaussian spatial tokens for 3d depth-aware VLA models. *arXiv preprint arXiv:2603.09079*, 2026. arXiv:2603.09079.

[11] R. Tu et al. SG-VLA: Learning spatially-grounded vision-language-action models for mobile manipulation. *arXiv preprint arXiv:2603.22760*, 2026. arXiv:2603.22760.

[12] J. Xiao et al. StemVLA: An open-source VLA with future 3d spatial geometry knowledge and 4d historical representation. *arXiv preprint arXiv:2602.23721*, 2026. arXiv:2602.23721.

[13] C. Li et al. PointVLA: Injecting the 3d world into vision-language-action models. *arXiv preprint arXiv:2503.07511*, 2025. arXiv:2503.07511.

[14] L. Sun et al. GeoVLA: Empowering 3d representations in vision-language-action models. *arXiv preprint arXiv:2508.09071*, 2025. arXiv:2508.09071.

[15] V. Bhat et al. 3D-CAVLA: Leveraging depth and 3d context to generalize VLA for unseen tasks. *arXiv preprint arXiv:2505.05800*, 2025. arXiv:2505.05800.

[16] Y. Li et al. QDepth-VLA: Quantized depth prediction as auxiliary supervision for vision-language-action models. *arXiv preprint arXiv:2510.14836*, 2025. arXiv:2510.14836.

[17] A. Abouzeid et al. GeoAware-VLA: Implicit geometry aware vision-language-action model. *arXiv preprint arXiv:2509.14117*, 2025. arXiv:2509.14117.

[18] Z. Rao et al. AugVLA-3D: Depth-driven feature augmentation for vision-language-action models. *arXiv preprint arXiv:2602.10698*, 2026. arXiv:2602.10698.

[19] D. Li et al. CorridorVLA: Explicit spatial constraints for generative action heads via sparse anchors. *arXiv preprint arXiv:2604.21241*, 2026. arXiv:2604.21241.

[20] X. Zhai et al. CoFreeVLA: Collision-free dual-arm manipulation via VLA model and risk estimation. *arXiv preprint arXiv:2601.21712*, 2026. arXiv:2601.21712.

[21] G. Raja, T. Mökkönen, and R. Ghabcheloo. Safe robot control using occupancy grid map-based control barrier function (OGM-CBF). *arXiv preprint arXiv:2405.10703*, 2024. arXiv:2405.10703.

[22] B. Zhang et al. SafeVLA: Towards safety alignment of vision-language-action model via constrained learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. Spotlight. arXiv:2503.03480.

[23] Q. Li et al. Vision-language-action safety: Threats, challenges, evaluations, and mechanisms. *arXiv preprint arXiv:2604.23775*, 2026. arXiv:2604.23775.

[24] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.

[25] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv:2401.10891.

[26] Z. Li, W. Wang, H. Li, E. Xie, C. Sima, T. Lu, Y. Qiao, and J. Dai. BEVFormer: Learning bird's-eye-view representation from multi-camera images via spatiotemporal transformers. In *European Conference on Computer Vision (ECCV)*, 2022. arXiv:2203.17270.

[27] C. Sima, W. Tong, T. Wang, L. Chen, S. Wu, H. Deng, Y. Gu, L. Lu, P. Luo, D. Lin, and H. Li. Scene as occupancy. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. arXiv:2306.02851.

[28] X. Tian, T. Jiang, L. Yun, Y. Mao, H. Yang, Y. Wang, Y. Wang, and H. Zhao. Occ3D: A large-scale 3d occupancy prediction benchmark for autonomous driving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2304.14365.

[29] R. Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997.

[30] S. Ruder. An overview of multi-task learning in deep neural networks. *arXiv preprint arXiv:1706.05098*, 2017. arXiv:1706.05098.

[31] Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich. GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *International Conference on Machine Learning (ICML)*, 2018. arXiv:1711.02257.

[32] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn. Gradient surgery for multi-task learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2001.06782.

[33] B. Liu, X. Liu, X. Jin, P. Stone, and Q. Liu. Conflict-averse gradient descent for multi-task learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2110.14048.

[34] A. Navon, A. Shamsian, I. Achituve, H. Maron, K. Kawaguchi, G. Chechik, and E. Fetaya. Multi-task learning as a bargaining game. In *International Conference on Machine Learning (ICML)*, 2022. arXiv:2202.01017.

[35] J. Jiang, B. Chen, J. Pan, X. Wang, D. Liu, J. Jiang, and M. Long. ForkMerge: Mitigating negative transfer in auxiliary-task learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2301.12618.

[36] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, 2015. arXiv:1502.03167.

[37] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. arXiv:1607.06450.

[38] Y. Wu and K. He. Group normalization. In *European Conference on Computer Vision (ECCV)*, 2018. arXiv:1803.08494.

[39] Y. Li, N. Wang, J. Shi, J. Liu, and X. Hou. Revisiting batch normalization for practical domain adaptation. In *International Conference on Learning Representations Workshop (ICLR Workshop)*, 2017. arXiv:1603.04779.

[40] S. Schneider, E. Rusak, L. Eck, O. Bringmann, W. Brendel, and M. Bethge. Improving robustness against common corruptions by covariate shift adaptation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[41] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jégou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2304.07193.

[42] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. arXiv:2303.15343.

[43] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2017. arXiv:1708.02002.

[44] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.03310.

[45] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 3519–3529, 2019.

[46] S. Lippl and J. W. Lindsey. Implicit regularization of multi-task learning and finetuning in overparameterized neural networks. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.02396.

[47] J. Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.

A Extended Per-Task Results

Table 4 reports per-task mean CAR across seeds for each condition. Seven tasks achieve CAR $\geq$ 0.92 under Vanilla, while three physically constrained tasks (T3, T5, T6) fall below 0.58. The overall CAR differences reported in the main text are dominated by these three hard tasks, which average 0.342 compared to 0.968 for the seven easy tasks.

Table 4: Per-task mean CAR across seeds (50 rollouts per seed, v1 code). This table reports v1 codebase results retained for completeness; all main analyses use v2 data (Table 2). All conditions use 3 seeds except Depth (4). Detach+LN per-task uses 2 seeds (s1, s2; s0 lacks per-task breakdown).

		Mean CAR by Condition						
Task	Diff.	Vanilla	Full OCC	Random	Detach	Det+LN	Depth	Cut@35K
T0	Easy	0.98	0.98	0.99	0.97	0.97	0.99	0.98
T1	Easy	0.92	0.93	0.99	0.89	0.78	0.93	0.95
T2	Easy	0.92	0.93	0.94	0.93	0.90	0.90	0.94
T3	Hard	0.38	0.37	0.31	**0.11**	0.44	0.50	0.55
T4	Easy	0.98	0.99	0.95	0.98	0.96	0.92	0.97
T5	Hard	0.07	0.17	0.23	**0.05**	0.12	0.08	0.04
T6	Hard	0.57	0.61	0.55	0.56	0.36	0.54	0.38
T7	Easy	0.97	0.99	0.95	0.98	1.00	0.96	0.98
T8	Easy	1.00	1.00	1.00	0.99	0.99	0.99	0.99
T9	Easy	1.00	0.99	0.99	1.00	1.00	1.00	1.00
Overall		0.780	0.797	0.789	0.746	0.765	0.780	0.779

Ceiling effects. On the seven easy tasks (T0–T2, T4, T7–T9), all conditions achieve CAR $\geq$ 0.89, leaving minimal room for auxiliary training to improve performance. The obstacle is spatially separated from the manipulation path in these tasks.

Hard task patterns. On Task 5 (Vanilla CAR 0.07), gradient-flow conditions (Random OCC 0.23, Full OCC 0.17) exceed Vanilla and Detach (0.05), but all values remain below 0.23. Cutoff@35K achieves the highest T3 CAR (0.55) but the lowest T5 CAR (0.04), which illustrates task-specific temporal sensitivity. Depth exhibits high cross-seed variance on hard tasks (T3 range: 0.20–0.78; T6 range: 0.16–0.84), consistent with auxiliary supervision increasing seed sensitivity.

B Training Details

Evaluation protocol. Each condition uses 3+ seeds (Depth uses 4; Cutoff@15K, Cutoff@25K, and Reverse Cutoff use 2; Cutoff@45K uses 1) with 50 rollouts per task per seed (500 episodes per seed). A collision is registered when the L1 displacement of the obstacle's `body_xpos` exceeds 10^{-3}. Bootstrap 95% CIs are computed via 10,000 resamples (percentile method).

Statistical tests. Pairwise comparisons against Vanilla use two-tailed permutation tests on per-seed mean CARs with Cohen's d for effect size. With n=3 seeds, statistical power is limited; we report p-values and effect sizes for transparency. **CKA analysis.** Centered Kernel Alignment [45] measures representational similarity between fine-tuned and pretrained encoder features. We extract patch token activations from the final layer of DINOv2 and SigLIP for 1,000 validation images, compute linear CKA per layer, and report the mean across layers. Higher CKA indicates better preservation of pretrained representations.

Cutoff implementation. As described in Section 3.1, setting $\lambda(t)$=0 does not disable the auxiliary head's forward pass; the post-cutoff phase operates in Detach mode with BatchNorm statistics continuing to update (see the Cutoff description therein for details).

Table 5: Training hyperparameters.

Parameter	Value
Base model	OpenVLA-7B [2]
Fine-tuning recipe	OFT [3]
Vision encoders	DINOv2 (1024-d) + SigLIP (1152-d)
Patch token dim.	2176 (concatenated)
LoRA rank / α	32 / 16
LoRA trainable params	110.8M (1.45% of 7.65B)
Optimizer	AdamW (weight decay 0.01)
Batch size	8
Learning rate	5×10^{-4} (constant)
Training steps	50,000
Aux. loss weight λ_0	0.01
Aux. loss decay	$\exp(-t/50{,}000)$
BEV grid resolution	64×64
Focal loss α / γ	0.25 / 2.0
Depth loss	Smooth-L1
Aux. head parameters	1.2M
Hardware	NVIDIA RTX PRO 6000 (96 GB)
Wall time per run	$\sim$12.9 hours

C Extended Analysis

The Detach+LN ablation replaces BatchNorm2d with LayerNorm in the auxiliary head while retaining `stop_gradient`. Under v1 (3 seeds), Detach+LN achieves CAR 0.765 ± 0.023; under v2 (n=3), Detach+LN achieves 0.802 ± 0.024 (Table 2), the highest factorial cell mean ($\Delta = +5.7$ pp vs. Vanilla, $d = +2.36$). This confirms the additive main effects: both detaching gradient flow and using LN independently improve safety.

Per-layer gradient analysis. Figure 4 decomposes auxiliary gradient flow into two dimensions: amplitude (top row) and direction (bottom row). The amplitude heatmap (panels a–b) shows that Vanilla, Full OCC, and Random OCC maintain comparable action gradient norms across all layers (warm colors), while Detach shows substantially reduced norms across both encoders (dark). These amplitude results were generated from v1 codebase checkpoints. The v2 gradient analysis (Figure 3) confirms that the Detach gradient suppression does not reproduce: Detach and gradient-flow conditions maintain comparable action gradient norms under v2, indicating the v1 suppression was a code-path artifact (see Appendix E). The directional analysis (panels c–d) reveals encoder-specific alignment: DINOv2 conflict rates hover near 46% across all layers while SigLIP late layers (blocks 21–24) drop to 19–25%, which indicates cooperative gradient alignment in semantically rich layers. Both patterns are identical for Full OCC and Random OCC, which confirms they arise from architectural structure rather than label content. The aggregate conflict rates (Full OCC 45.6%, Full OCC+LN 45.7% under v2; Full OCC 44.1%, Random OCC 43.9% under v1) fall below the 50% expected for random vectors because they reflect this mixture of two encoder-specific patterns.

C.1 Temporal Dynamics

Dose-response: gradient flow duration vs. CAR (v1 data). Figure 5 plots CAR against the number of OCC gradient flow steps for the cutoff series (15K–45K, plus Detach at 0K and Full OCC at 50K; all v1 code). Cutoff@15K (2 seeds, CAR 0.782) clusters near Vanilla (0.780), while Cutoff@25K (2 seeds, 0.724) falls substantially below. Multi-seed conditions (Cutoff@35K and Full OCC) also cluster near the Vanilla baseline. Reverse Cutoff (late 15K) falls below the early-cutoff counterpart at the same gradient flow duration, consistent with the continuity hypothesis. The pattern suggests intermediate cutoff $\approx$ Vanilla rather than a non-monotonic dose-response; the earlier single-seed Cutoff@15K result (0.804) was seed noise.

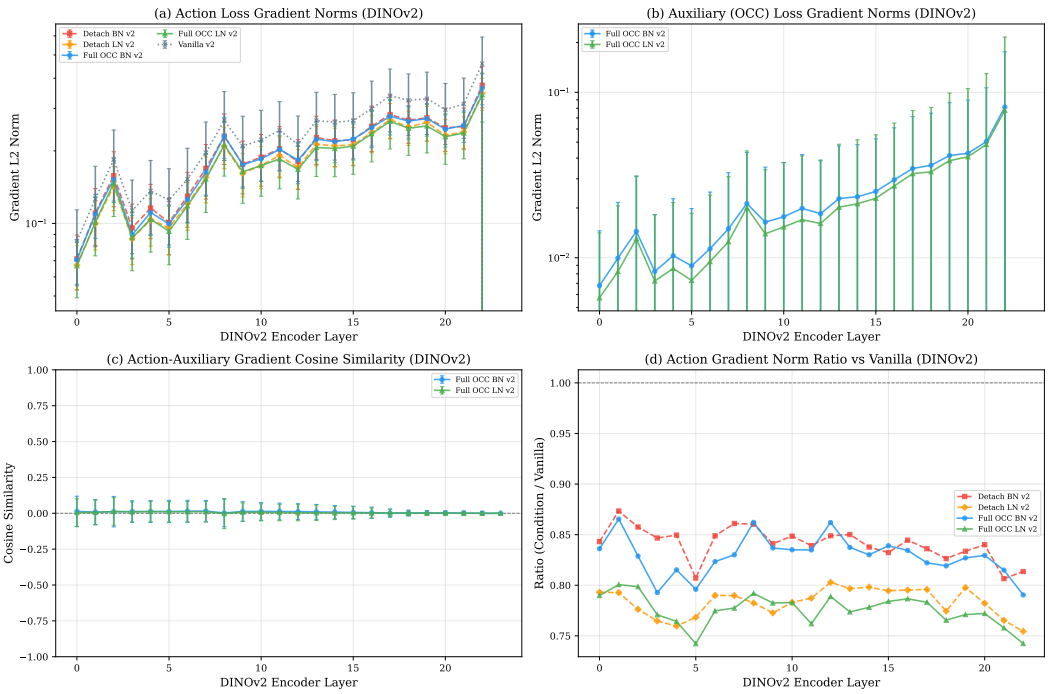


Figure 3: Gradient dynamics during VLA fine-tuning (v2 codebase; seed 42, 200 batches at 50K checkpoint). Five conditions: Vanilla, Full OCC (BN), Full OCC+LN, Detach OCC (BN), Detach+LN. (a) Action loss gradient norms per encoder layer: all conditions maintain comparable norms, ruling out gradient amplification. (b) Auxiliary loss gradient norms in the shared encoder (Detach conditions excluded; no auxiliary gradients by design). (c) Action–auxiliary gradient cosine similarity per layer (near zero for all gradient-flow conditions, confirming orthogonality; conflict rate $\approx$46%). (d) Gradient norm ratio per layer.

D Additional Observations

Per-task concentration. Seven of ten tasks achieve CAR ≥ 0.92 under Vanilla; overall CAR is dominated by three physically constrained tasks (T3, T5, T6; Section A). Under the v1 codebase, Detach showed the lowest CAR on these hard tasks (T3: 0.11, T5: 0.05). However, this v1-specific pattern does not replicate under the controlled v2 factorial design (Table 2), where detaching gradient flow consistently preserves safety (Detach OCC CAR 0.767 vs. Vanilla 0.745); the v1 per-task pattern may reflect implementation-specific artifacts rather than a generalizable effect of gradient detachment.

Egocentric depth matches Vanilla (v1 data). Depth (4 seeds) achieves CAR 0.780 ± 0.039, identical to Vanilla (v1, 0.780), despite providing per-pixel geometric information. Three label types (structured BEV, random binary, egocentric depth) yield comparable CAR when gradient flow is maintained.

Temporal dynamics (preliminary, v1 data). Cutoff@35K (3 seeds) achieves CAR 0.779, statistically indistinguishable from Vanilla (v1, $\Delta = -0.1$ pp), which confirms that 35K steps of gradient flow fully protects collision avoidance. Surprisingly, Cutoff@25K (0.724, 2 seeds) and Reverse Cutoff (0.718, 2 seeds) fall substantially *below* Vanilla. The most parsimonious explanation: setting the loss weight to zero retains the auxiliary head forward pass, so the post-cutoff period operates in Detach mode; BatchNorm interference acts on representations shaped by gradient flow, without gradient compensation. This is not withdrawal from gradient flow *per se*, but transition into an actively harmful regime. Third seeds are in progress; extended analysis in Section C.1.

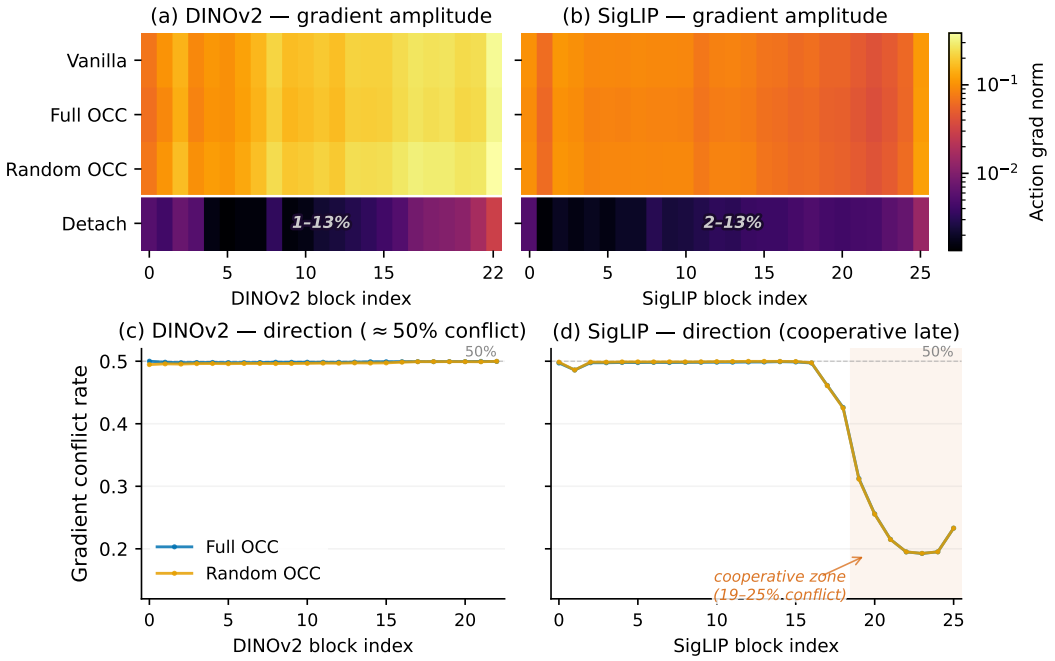


Figure 4: Per-layer gradient analysis decomposed into amplitude and direction (v1 codebase checkpoint, seed 42, 200 batches). **Top (a–b):** Action gradient norm heatmap (log scale). Gradient-flow conditions (Vanilla, Full OCC, Random OCC) maintain comparable norms; Detach shows substantially reduced norms across all layers, a v1-specific artifact not reproduced under v2 (see Appendix E). **Bottom (c–d):** Gradient conflict rate between action and auxiliary losses. DINOv2 maintains ∼46% conflict (near-orthogonal), while SigLIP late layers drop to 19–25% (cooperative alignment). Both amplitude and directional patterns are identical for Full OCC and Random OCC, confirming architectural rather than label-dependent structure. Directional findings are confirmed under v2; the Detach amplitude suppression (a–b) is a v1 code-path artifact. Conflict rates approach ≈46% in early and middle layers; late SigLIP layers show lower conflict rates (≈25%), consistent with weak alignment rather than conflict.

Cross-benchmark Vanilla baseline. Vanilla baseline on LIBERO-Object achieves CAR **X.XX** (n=2), confirming that the safety gap between TSR and CAR is not specific to LIBERO-Spatial.

E Codebase Sensitivity (v1 vs. v2)

The five core factorial conditions were retrained under a revised codebase (v2) that fixed a batch-index resume bug affecting cutoff timing and restructured the auxiliary head pipeline (module initialization order, data loading). All model architectures, hyperparameters, LoRA configuration, and evaluation protocols are identical between v1 and v2. The Vanilla baseline (no auxiliary head) is unaffected by these changes, since the revision touches only auxiliary-head code paths.

Behavioral shift. Under v1, Full OCC achieved a mean CAR of 0.797 (3 seeds, per-task Table 4); under v2, the same condition yields 0.725 ± 0.064 (Table 2). Same-seed comparisons show shifts of up to 8.6 pp, while Vanilla remains stable across codebases (<1.5 pp). This sensitivity is confined to conditions with auxiliary heads and is itself informative: it indicates that auxiliary head training dynamics are fragile with respect to implementation details that would not normally be considered hyperparameters.

Gradient analysis shift. The v1 codebase exhibited a Detach gradient suppression artifact: Detach action-loss gradient norms were only ∼4% of Vanilla norms across all encoder layers. In v2, this suppression disappears entirely (Detach/Vanilla ratio ≈ 1.29), indicating that the v1 pattern was a code-path artifact rather than a genuine effect of gradient detachment. The gradient conflict rate

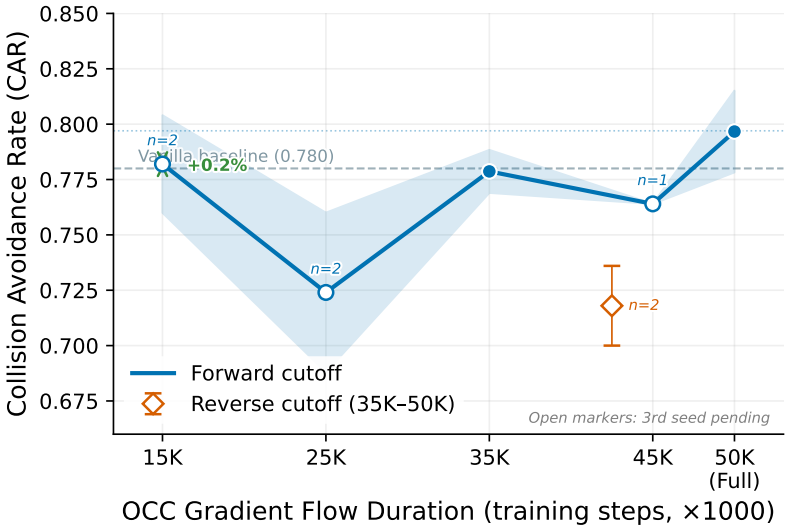


Figure 5: Dose-response: CAR vs. OCC gradient flow duration (v1 codebase, all conditions). Filled circles: early-cutoff conditions with all 3 seeds completed; open circles: conditions with $\leq$2 seeds (counts annotated as n). Diamond ($\diamond$): Reverse Cutoff (gradient flow restricted to the last 15K steps). Dashed line: Vanilla 3-seed mean (v1); shaded band: $\pm$1 SE. The "+0.2%" annotation marks the Cutoff@15K–Vanilla gap. Intermediate cutoff conditions cluster near Vanilla, while Cutoff@25K and Reverse Cutoff fall substantially below.

also shifted from $\sim$44% (v1) to $\sim$46% (v2), with v2 values closer to the theoretical expectation for orthogonal vectors.

Effect pattern shift. Under v1, the normalization $\times$ gradient flow pattern appeared as a crossover (p=0.025); under v2, both rows slope in the same direction, consistent with two additive main effects rather than an interaction (ANOVA interaction F<0.1, p=0.76, η^2=0.005). The crossover pattern in v1 is likely attributable to the Detach gradient suppression artifact described above, which differentially affected BN and LN conditions. All main-text analyses use v2 data exclusively; v1 results are retained in the appendix for completeness and to document the sensitivity.

Implication. The codebase sensitivity reinforces our central finding: auxiliary spatial heads introduce optimization fragility that implementation details can amplify or attenuate. This fragility is a further argument against adding auxiliary spatial heads for safety purposes, since the safety outcome depends on implementation choices that are difficult to specify, reproduce, or audit.

F Statistical Power Analysis

The 2$\times$2 factorial design prioritizes breadth (12 conditions, 5 factors) over per-cell statistical power. With n=3–4 per cell (14 total observations), the design yields medium power for the observed effect sizes. The normalization main effect approaches significance (p=0.060, η^2=0.253); the gradient flow main effect does not reach significance (p=0.104) but shows a medium-to-large effect size (η^2=0.180). We report effect sizes as the primary inferential quantity.

The study's evidential strength rests on the consistent direction of both main effects: both BN and LN rows show lower CAR under gradient flow ON than OFF, and both gradient flow conditions show higher CAR under LN than BN. The negligible interaction (η^2=0.005) confirms these effects are additive.

Formal 2$\times$2 ANOVA. Table 6 reports the Type II factorial analysis (n=3–4 per cell, 14 total observations). The interaction is negligible ($F(1, 10)$=0.10, p=0.763, η^2=0.005), confirming additive main effects. The normalization main effect approaches significance ($F(1, 10)$=4.49, p=0.060,

η^2=0.253); the gradient flow main effect is medium-sized but non-significant ($F(1, 10)$=3.20, p=0.104, η^2=0.180). The non-significant gradient flow p-value reflects limited per-cell power rather than an absent effect.

Table 6: 2×2 factorial ANOVA (Type II SS). The interaction is negligible; the normalization main effect approaches significance (p=0.060).

Source	F	df	p	η^2
Gradient flow (ON vs. OFF)	3.20	1, 10	0.104	0.180
Normalization (BN vs. LN)	4.49	1, 10	0.060	0.253
Interaction	0.10	1, 10	0.763	0.005

Pairwise effect sizes. Table 7 reports Cohen's d for all factorial cell pairs and versus Vanilla (n=3–4 per cell).

Table 7: Pairwise Cohen's d for factorial cells and Vanilla.

Comparison	Cohen's d
Cell 1 (Full OCC BN) vs. Cell 2 (Full OCC+LN)	−1.05
Cell 1 vs. Cell 3 (Detach OCC)	−0.93
Cell 1 vs. Cell 4 (Detach+LN)	−1.60
Cell 2 vs. Cell 3	+0.25
Cell 2 vs. Cell 4	−1.12
Cell 3 vs. Cell 4	−1.47
Cell 1 vs. Vanilla	−0.42
Cell 2 vs. Vanilla	+1.07
Cell 3 vs. Vanilla	+0.88
Cell 4 vs. Vanilla	+2.36

G Per-Task Factorial Analysis

The three physically constrained tasks (T3: `cookie_box`, T5: `ramekin`, T6: `next_to_cookie_box`) amplify both main effects by approximately 3× relative to the overall means.

Normalization effect amplification. The overall normalization effect ($\Delta = +3.5$–4.8 pp between BN and LN) is amplified on constrained tasks. The LN > BN direction is consistent across all three constrained tasks.

Gradient flow on constrained tasks. On T5 (ramekin), the best auxiliary condition versus Vanilla yields d=2.65, the largest per-task effect size observed. T5 also shows a reversed gradient flow direction ($d = +1.25$, ON > OFF), but this reflects near-floor performance (CAR 0.04–0.19) where absolute differences are small and floor artifacts are expected.

These per-task patterns suggest that the safety-relevant design choices (normalization type, gradient flow) are disproportionately consequential on physically constrained tasks where collision avoidance is most difficult.

H Limitations

Our study uses a single architecture (OpenVLA-OFT with LoRA rank 32); full fine-tuning may alter the interaction between auxiliary heads and encoder optimization because all parameters update simultaneously, potentially changing the gradient flow dynamics we observe. All experiments are conducted in MuJoCo simulation on a single task suite (SafeLIBERO Level I, 10 tasks drawn from LIBERO-Spatial); whether the normalization-dependent gradient flow effect generalizes to other task distributions remains open. LIBERO-Object, which involves different spatial constraints, does not provide ground-truth occupancy labels, preventing factorial comparison on that benchmark;

generating synthetic labels is straightforward but was not pursued in the current study. Real-world deployment introduces additional sources of distributional shift that could interact differently with auxiliary head training dynamics. The benchmark contains only static obstacles; dynamic obstacle scenarios may induce time-varying distributions where auxiliary head dynamics behave differently. Our collision metric is binary (L1 displacement threshold 10^{-3}), which does not capture collision severity or near-miss behavior; a continuous safety metric might reveal finer-grained differences between conditions. CKA measures linear representational similarity, which may miss nonlinear structure relevant to safety behavior. The factorial design uses n=3–4 seeds per cell (14 total observations), which limits statistical power for individual pairwise tests; we report effect sizes as the primary inferential quantity rather than relying on p-values. Larger-scale replication would further strengthen conclusions. Cutoff conditions have 1–3 seeds, limiting temporal conclusions to preliminary observations. All auxiliary conditions use a fixed loss weight λ_0=0.01 with exponential decay; we did not sweep this hyperparameter, so the gradient flow effect may vary under different loss weight schedules.